# Torque Duty Cycle

*Component Design Document*

## 1 Description

Duty cycle for a commanded body torque. Passes the torque through for the first on periods of every cycle and outputs zero for the remaining off periods. In the desaturation state it sits between momentum management and the thruster force mapping, so the thrusters fire in pulses with quiet windows in which the reaction wheels re-stabilize the attitude. The cadence is free-running and advances every tick. Wraps the VectorDutyCycleAlgorithm C++ algorithm via its C shim, which it shares with the force duty cycle.

## 2 Requirements

No requirements have been specified for this component.

## 3 Design

### 3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *passive*
- **Number of Connectors** - 5
- **Number of Invokee Connectors** - 3
- **Number of Invoker Connectors** - 2
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - *None*
- **Number of Parameters** - 2
- **Number of Events** - *None*
- **Number of Faults** - *None*
- **Number of Data Products** - 1
- **Number of Data Dependencies** - 1
- **Number of Packets** - *None*

## 3.2 Diagram

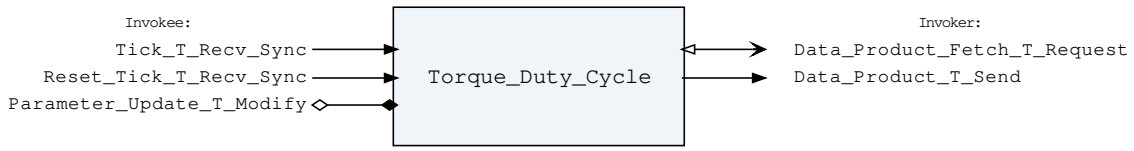



Figure 1: Torque Duty Cycle component diagram.

## 3.3 Connectors

Below are tables listing the component's connectors.

### 3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Torque Duty Cycle Invokee Connectors

| Name | Kind | Type | Return_Type | Count |
|---|---|---|---|---|
| Tick_T_Recv_Sync | recv_sync | Tick.T | - | 1 |
| Reset_Tick_T_ Recv_Sync | recv_sync | Tick.T | - | 1 |
| Parameter_ Update_T_Modify | modify | Parameter_ Update.T | - | 1 |

Connector Descriptions:

- **Tick_T_Recv_Sync** - Apply one control period of the duty cycle to the commanded torque.
- **Reset_Tick_T_Recv_Sync** - Restart the duty cycle at the beginning of its on window. The assembly fires this on a state change.
- **Parameter_Update_T_Modify** - The parameter update connector.

### 3.3.2 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 2: Torque Duty Cycle Invoker Connectors

| Name | Kind | Type | Return_Type | Count |
|---|---|---|---|---|
| Data_Product_ Fetch_T_Request | request | Data_Product_ Fetch.T | Data_Product_ Return.T | 1 |
| Data_Product_T_ Send | send | Data_Product.T | - | 1 |

Connector Descriptions:

- **Data_Product_Fetch_T_Request** - Fetch a data product item from the database.
- **Data_Product_T_Send** - The data product invoker connector

## 3.4 Interrupts

This component contains no interrupts.

## 3.5 Initialization

Below are details on how the component should be initialized in an assembly.

### 3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

### 3.5.2 Component Base Initialization

This component contains no base class initialization, meaning there is no `init_Base` subprogram for this component.

### 3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 3: Torque Duty Cycle Set Id Bases Parameters

| Name | Type |
|---|---|
| `Data_Product_Id_Base` | `Data_Product_Types.Data_Product_Id_Base` |
| `Parameter_Id_Base` | `Parameter_Types.Parameter_Id_Base` |

Parameter Descriptions:

- **`Data_Product_Id_Base`** - The value at which the component's data product identifiers begin.
- **`Parameter_Id_Base`** - The value at which the component's parameter identifiers begin.

### 3.5.4 Component Map Data Dependencies

This component contains data dependencies that need their IDs mapped to valid data product IDs at initialization. Each data dependency also needs a stale limit provided in microseconds. If the timestamp of the data dependency is older than the reference time (usually the current time) minus the stale limit then the data dependency is considered too old to safely use. The `map_Data_Dependencies` procedure must be called with the following parameters:

Table 4: Torque Duty Cycle Map Data Dependencies Parameters

| Name | Type |
|---|---|
| `Commanded_Torque_Id` | `Data_Product_Types.Data_Product_Id` |
| `Commanded_Torque_Stale_Limit` | `Ada.Real_Time.Time_Span` |

Parameter Descriptions:

- **`Commanded_Torque_Id`** - The external data product ID used to fetch the value for the 'Commanded_Torque' data dependency.
- **`Commanded_Torque_Stale_Limit`** - The stale limit used for the 'Commanded_Torque' data dependency. If the fetched data product is older than the given Time_Span then it is deemed stale and unfit for use by the component.

### 3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. Initializes the duty cycle with the default parameter values. The `init` subprogram takes no parameters.

## 3.6 Commands

The Torque Duty Cycle component has no commands.

## 3.7 Parameters

Parameters for the Torque Duty Cycle component.

Table 5: Torque Duty Cycle Parameters

| Local ID | Parameter Name | Type | Default Value |
|---|---|---|---|
| 0x0000 (0) | On_Periods | Packed_U32.T | (Value=>1) |
| 0x0001 (1) | Off_Periods | Packed_U32.T | (Value=>0) |

Parameter Descriptions:

- **On_Periods** - [-] Control periods at the start of each cycle in which the output equals the input vector. Zero keeps the output at zero and needs at least one off period.
- **Off_Periods** - [-] Control periods in which the output is zero. Any value whose sum with On_Periods is at least 1 and fits in 32 bits. Zero disables the duty cycle.

## 3.8 Events

The Torque Duty Cycle component has no events.

## 3.9 Data Products

Data products for the Torque Duty Cycle component.

Table 6: Torque Duty Cycle Data Products

| Local ID | Data Product Name | Type |
|---|---|---|
| 0x0000 (0) | Gated_Torque | Cmd_Torque_Body.T |

Data Product Descriptions:

- **Gated_Torque** - [Nm] The commanded torque during an on period, zero during an off period. The thruster force mapping takes this as its commanded torque.

## 3.10 Data Dependencies

Data dependencies for the Torque Duty Cycle component.

Table 7: Torque Duty Cycle Data Dependencies

| Data Dependency Name | Type |
|---|---|
| Commanded_Torque | Cmd_Torque_Body.T |

Data Dependency Descriptions:

- **Commanded_Torque** - [Nm] The body torque requested by momentum management, the vector the duty cycle applies to.

### 3.11 Packets

The Torque Duty Cycle component has no packets.

### 3.12 Faults

The Torque Duty Cycle component has no faults.

## 4 Unit Tests

The following section describes the unit test suites written to test the component.

### 4.1 *Torque_Duty_Cycle_Tests* Test Suite

This is a unit test suite for the Torque Duty Cycle component

Test Descriptions:

- **`Test`** - Run the cadences of the Python reference test to ensure the Ada to C to C++ integration is sound.
- **`Test_Reset`** - Check that the reset connector applies a pending cadence and restarts the duty cycle at its on window, and that a parameter update alone does neither.
- **`Test_Invalid_Parameter`** - Ensure a staged configuration the algorithm would reject is refused at validation.
- **`Test_Invalid_Data_Dependency`** - Ensure a data dependency with the wrong identifier is treated as a wiring defect and fails the tick's assertion.

## 5 Appendix

### 5.1 Preamble

This component contains no preamble code.

### 5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

### Cmd_Torque_Body.T:

Commanded control torque in body frame components, as produced by the MRP control laws.

Table 8: Cmd_Torque_Body Packed Record : 96 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Torque_Request_Body | Packed_F32x3.T | - | 96 | 0 | 95 |

Field Descriptions:

- **`Torque_Request_Body`** - [Nm] Control torque requested

### Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 9: Data_Product Packed Record : 1096 bits *(maximum)*

| Name | Type | Range | Size (Bits) | Start Bit | End Bit | Variable Length |
|---|---|---|---|---|---|---|
| Header | Data_Product_ Header.T | - | 72 | 0 | 71 | - |
| Buffer | Data_Product_ Types.Data_ Product_ Buffer_Type | - | 1024 | 72 | 1095 | Header. Buffer_Length |

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

## Data_Product_Fetch.T:

A packed record which holds information for a data product request.

Table 10: Data_Product_Fetch Packed Record : 16 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Id | Data_Product_Types. Data_Product_Id | 0 to 65535 | 16 | 0 | 15 |

Field Descriptions:

- **Id** - The data product identifier

## Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 11: Data_Product_Header Packed Record : 72 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Time | Sys_Time.T | - | 48 | 0 | 47 |
| Id | Data_Product_Types. Data_Product_Id | 0 to 65535 | 16 | 48 | 63 |
| Buffer_Length | Data_Product_ Types.Data_Product_ Buffer_Length_Type | 0 to 128 | 8 | 64 | 71 |

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

## Data_Product_Return.T:

This record holds data returned from a data product fetch request.

Table 12: Data_Product_Return Packed Record : 1104 bits *(maximum)*

| Name | Type | Range | Size (Bits) | Start Bit | End Bit | Variable Length |
|---|---|---|---|---|---|---|
| The_Status | Data_Product_Enums.Fetch_Status.E | 0 => Success<br>1 => Not_Available<br>2 => Id_Out_Of_Range | 8 | 0 | 7 | - |
| The_Data_Product | Data_Product.T | - | 1096 | 8 | 1103 | - |

Field Descriptions:

- **The_Status** - A status relating whether or not the data product fetch was successful or not.
- **The_Data_Product** - The data product item returned.

## Packed_F32x3.T:

Packed 3 element vector of 32-bit floats.

Table 13: Packed_F32x3 Packed Array : 96 bits

| Type | Range | Element Size (Bits) | Length | Total Size (Bits) |
|---|---|---|---|---|
| **Short_Float** | -3.40282e+38 to 3.40282e+38 | 32 | 3 | 96 |

## Packed_U32.T:

Single component record for holding packed unsigned 32-bit value.

Table 14: Packed_U32 Packed Record : 32 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Value | Interfaces.Unsigned_32 | 0 to 4294967295 | 32 | 0 | 31 |

Field Descriptions:

- **Value** - The 32-bit unsigned integer.

## Parameter.T:

Generic parameter packet for holding a generic parameter

Table 15: Parameter Packed Record : 792 bits *(maximum)*

| Name | Type | Range | Size (Bits) | Start Bit | End Bit | Variable Length |
|---|---|---|---|---|---|---|
| Header | Parameter_Header.T | - | 24 | 0 | 23 | - |

| | | | | | | |
|---|---|---|---|---|---|---|
| Buffer | Parameter_ Types. Parameter_ Buffer_Type | - | 768 | 24 | 791 | Header.Buffer_ Length |

Field Descriptions:

- **Header** - The parameter header
- **Buffer** - A buffer that contains the parameter type

## Parameter_Header.T:

Generic parameter header for holding arbitrary parameters

Table 16: Parameter_Header Packed Record : 24 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Id | Parameter_Types. Parameter_Id | 0 to 65535 | 16 | 0 | 15 |
| Buffer_Length | Parameter_Types. Parameter_Buffer_ Length_Type | 0 to 96 | 8 | 16 | 23 |

Field Descriptions:

- **Id** - The parameter identifier
- **Buffer_Length** - The number of bytes used in the parameter type buffer

## Parameter_Update.T:

A record intended to be used as a provide/modify connector type for updating/fetching parameters.

Table 17: Parameter_Update Packed Record : 824 bits *(maximum)*

| Name | Type | Range | Size (Bits) | Start Bit | End Bit | Variable Length |
|---|---|---|---|---|---|---|
| Table_Id | Parameter_ Types. Parameter_ Table_Id | 0 to 65535 | 16 | 0 | 15 | - |
| Operation | Parameter_ Enums. Parameter_ Operation_ Type.E | 0 => Stage<br>1 => Update<br>2 => Fetch<br>3 => Validate<br>4 => Discard | 8 | 16 | 23 | - |
| Status | Parameter_ Enums. Parameter_ Update_ Status.E | 0 => Success<br>1 => Id_Error<br>2 => Validation_Error<br>3 => Length_Error | 8 | 24 | 31 | - |

| | | | | | | |
|---|---|---|---|---|---|---|
| Param | Parameter.T | - | 792 | 32 | 823 | - |

Field Descriptions:

- **Table_Id** - The ID for the table that contains this parameter
- **Operation** - The parameter operation to perform.
- **Status** - The parameter return status.
- **Param** - The parameter that has been updated or fetched.

### Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 18: Sys_Time Packed Record : 48 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Seconds | Interfaces.Unsigned_32 | 0 to 4294967295 | 32 | 0 | 31 |
| Subseconds | Interfaces.Unsigned_16 | 0 to 65535 | 16 | 32 | 47 |

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of 1/(2^16) sub-seconds.

### Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 19: Tick Packed Record : 80 bits

| Name | Type | Range | Size (Bits) | Start Bit | End Bit |
|---|---|---|---|---|---|
| Time | Sys_Time.T | - | 48 | 0 | 47 |
| Count | Interfaces.Unsigned_32 | 0 to 4294967295 | 32 | 48 | 79 |

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

## 5.3 Enumerations

The following section outlines any enumerations used in the component.

### Data_Product_Enums.Fetch_Status.E:

This status denotes whether a data product fetch was successful.

Table 20: Fetch_Status Literals:

| Name | Value | Description |
|---|---|---|
| Success | 0 | The data product was returned successfully. |
| Not_Available | 1 | No data product is yet available for the provided id. |
| Id_Out_Of_Range | 2 | The data product id was out of range. |

## Parameter_Enums.Parameter_Operation_Type.E:

This enumeration lists the different parameter operations that can be performed.

Table 21: Parameter_Operation_Type Literals:

| Name | Value | Description |
|---|---|---|
| Stage | 0 | Stage the parameter. |
| Update | 1 | All parameters are staged, it is ok to update all parameters now. |
| Fetch | 2 | Fetch the parameter. |
| Validate | 3 | Validate all the parameters. |
| Discard | 4 | Discard any parameters staged by this table that have not been updated. |

## Parameter_Enums.Parameter_Update_Status.E:

This status enumeration provides information on the success/failure of a parameter operation.

Table 22: Parameter_Update_Status Literals:

| Name | Value | Description |
|---|---|---|
| Success | 0 | Parameter was successfully staged. |
| Id_Error | 1 | Parameter id was not valid. |
| Validation_Error | 2 | Parameter values were not successfully validated. |
| Length_Error | 3 | Parameter length was not correct. |